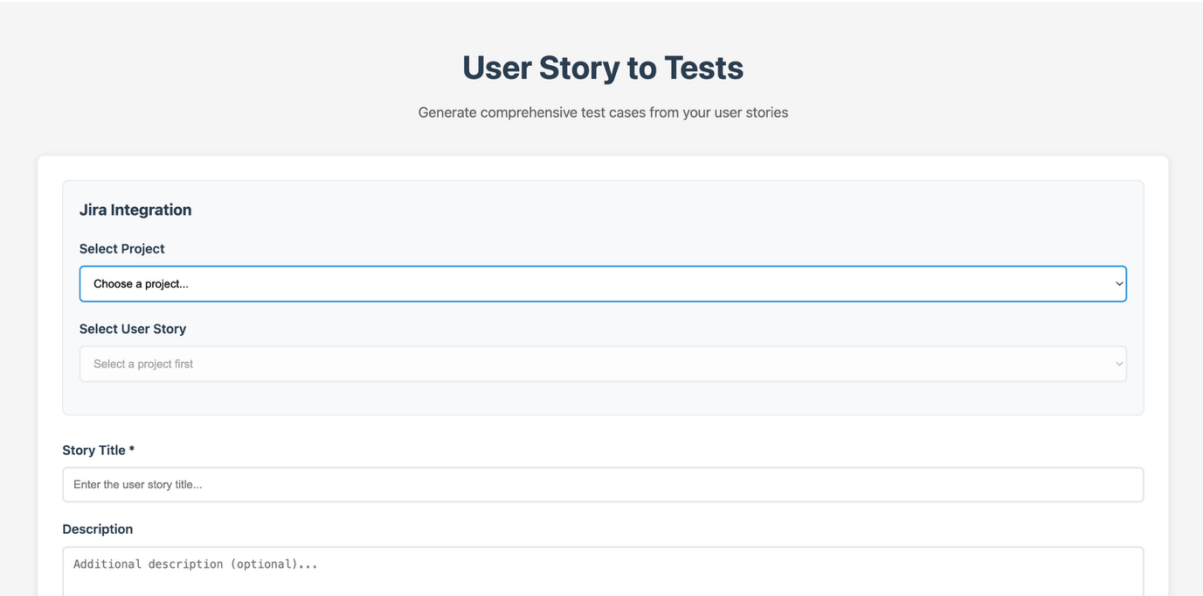
Week 03 – Day 02 Assignment output

Frontend screen after vibecoding:



I now can see the “Jira Integration” section at the top of the page and can see a project (I am currently not seeing any projects in the list, but this is the same with postman as well).

I am sure that once I get the postman collection working with the correct response, this will automatically show my projects and stories in the dropdown. I suspect the issue is with the way that Jira has setup my project (as spaces instead of projects), since it did NOT give me any option to create a SCRUM project during sign up.

Github Copilot output (for reference):

Step 1: Analysis of Existing Codebase

I've analyzed your existing codebase. Here's a summary:

Backend (Node.js + TypeScript + Express)

- **Structure**: Express server with CORS, dotenv for env vars, and a single route for generating tests via Groq LLM.
- **Current Functionality**: Accepts user story details (title, acceptance criteria, description, additional info) and generates test cases using AI.
- **Dependencies**: Already has `axios` for HTTP requests, which is perfect for Jira API calls.
- **Environment**: Loads .env from root directory, with variables like groq_API_KEY, PORT, etc.

Frontend (React + TypeScript + Vite)

- **Structure**: Simple React app with a form for user story input and test case display.
- **Current Functionality**: Form with fields for story title, acceptance criteria, description, and additional info. Displays generated test cases with download options.
- **API Integration**: Calls backend `/api/generate-tests` endpoint.

Postman Collection Reference

- Uses Jira Cloud API (`https://shaneagle21ai3.atlassian.net/rest/api/3/`)
- Authentication: Basic auth with email and API token
- Endpoints:
  - Projects: Can use `/rest/api/3/project` to fetch all accessible projects
  - User Stories: `/rest/api/3/search/jql` with JQL like `project = SCRUM AND issuetype = Story`

Proposed Implementation Plan

Backend Changes:

1. Add Jira env vars: `JIRA_EMAIL`, `JIRA_API_KEY`, `JIRA_BASE_URL`
2. Create Jira client/service for API calls
3. Add routes: `/api/jira/projects` and `/api/jira/user-stories/:projectKey`
4. Return project list and user story list with key, summary, description

Frontend Changes:

1. Add project dropdown (fetch on load)
2. Add user story dropdown (fetch when project selected)
3. Auto-populate form fields when user story selected
4. Update types and API calls

Security Note: API key and email will be configurable via environment variables, not hardcoded.

Approval Required: Do you approve proceeding to Step 2 (Backend Implementation)? If yes, I'll start implementing the Jira connectivity and routes in the backend.

Approved. Proceed to step 2.

Reviewed 2 files

Updated .env and modified 3 files

Ran `cd /Users/stephenprasad/Documents/MyStuff/Learning...`

Ran `cd /Users/stephenprasad/Documents/MyStuff/Learning...`

```
cd /Users/stephenprasad/Documents/MyStuff/Learning/Tech/TestLeaf/Week3Day2-files/user-story-to-tests/backend && npm run typecheck
```

```
npm error Lifecycle script `typecheck` failed with error:
npm error code 2
npm error path /Users/stephenprasad/Documents/MyStuff/Learning/Tech/T
k3Day2-files/user-story-to-tests/backend
npm error workspace backend@1.0.0
npm error location /Users/stephenprasad/Documents/MyStuff/Learning/Te
```

Step 2: Backend Implementation - Complete ✅

I've successfully implemented the Jira integration in the backend. Here's what was added:

Changes Made:

1. **Environment Variables** (.env):
   - Added JIRA_EMAIL , JIRA_API_KEY , JIRA_BASE_URL for configurable Jira connectivity
   - Updated server debug logs to show Jira configuration status
2. **Jira Client** (jiraClient.ts):
   - Created `JiraClient` class using axios with basic authentication
   - Methods: `getProjects()` and `getUserStories(projectKey)`
   - Uses JQL queries matching your Postman collection: `project = "KEY" AND issuetype = Story`
   - Includes proper error handling and validation
3. **Jira Routes** (jira.ts):
   - GET /api/jira/projects - Fetches all accessible Jira projects
   - `GET /api/jira/user-stories/:projectKey` - Fetches user stories for a specific project
   - Integrated into main server with proper error handling
4. **Type Safety** (schemas.ts):
   - Added Zod schemas for Jira project and user story responses
   - Exported TypeScript types for frontend consumption
   - Validation ensures API responses match expected structure
5. **Server Integration** (server.ts):
   - Imported and mounted Jira routes at `/api/jira`
   - Updated environment variable logging

API Endpoints:

- GET /api/jira/projects → Returns { projects: JiraProject[] }
- `GET /api/jira/user-stories/SCRUM` → Returns { userStories: JiraUserStory[] }

```
/week3day2-files/user-story-to-tests/backend
npm error command failed
npm error command sh -c tsc --noEmit
```

Reviewed and updated tsconfig.json and ran command

Ran `npm run typecheck`

Ran `npm run build`

Ran `timeout 10s npm run dev`

```
timeout 10s npm run dev
```

Ran `npm run dev`

Killed terminal

Step 2: Backend Implementation - Complete ✅

I've successfully implemented the Jira integration in the backend. Here's what was added:

Changes Made:

1. **Environment Variables** (.env):
   - Added JIRA_EMAIL , JIRA_API_KEY , JIRA_BASE_URL for configurable Jira connectivity
   - Updated server debug logs to show Jira configuration status
2. **Jira Client** (jiraClient.ts):
   - Created `JiraClient` class using axios with basic authentication
   - Methods: `getProjects()` and `getUserStories(projectKey)`
   - Uses JQL queries matching your Postman collection: `project = "KEY" AND issuetype = Story`
   - Includes proper error handling and validation
3. **Jira Routes** (jira.ts):
   - GET /api/jira/projects - Fetches all accessible Jira projects
   - `GET /api/jira/user-stories/:projectKey` - Fetches user stories for a specific project
   - Integrated into main server with proper error handling
4. **Type Safety** (schemas.ts):
   - Added Zod schemas for Jira project and user story responses
   - Exported TypeScript types for frontend consumption
   - Validation ensures API responses match expected structure
5. **Server Integration** (server.ts):
   - Imported and mounted Jira routes at `/api/jira`
   - Updated environment variable logging

API Endpoints:

- GET /api/jira/projects → Returns { projects: JiraProject[] }
- `GET /api/jira/user-stories/SCRUM` → Returns { userStories: JiraUserStory[] }

Configuration Required:

Update .env with your actual Jira credentials:

```
JIRA_EMAIL=your-actual-email@example.com
JIRA_API_KEY=your-actual-api-key
JIRA_BASE_URL=https://your-domain.atlassian.net
```

Validation:

- ✅ TypeScript compilation successful
- ✅ Build successful
- ✅ Server starts correctly (environment variables loaded)
- ✅ All dependencies available (axios already in package.json)

Approval Required: Do you approve proceeding to Step 3 (Frontend Implementation)? If yes, I'll implement the UI changes to add project and user story dropdowns that auto-populate from the new backend endpoints.

Approved. Proceed with Step 3.

Reviewed and updated types.ts and App.tsx

Ran `cd /Users/stephenprasad/Documents/MyStuff/Learning...`

Ran `npm run build`

Ran `cd /Users/stephenprasad/Documents/MyStuff/Learning...`

Updated .env and ran command

Edited .env +1 -1

Ran `cd /Users/stephenprasad/Documents/MyStuff/Learning...`

```
cd /Users/stephenprasad/Documents/MyStuff/Learning/Tech/TestLeaf/Week3Day2-files/user-story-to-tests && timeout 15s npm run dev
```

Ran `npm run dev`

Killed terminal

Step 3: Frontend Implementation - Complete ✅

I've successfully implemented the Jira integration in the frontend. Here's what was added:

Changes Made:

1. **Type Definitions** (`types.ts`):
   - Added `JiraProject`, `JiraUserStory`, `JiraProjectsResponse`, `JiraUserStoriesResponse` interfaces
   - Match the backend schema definitions for type safety
2. **API Functions** (`api.ts`):
   - `fetchJiraProjects()` - Calls `/api/jira/projects` endpoint
   - `fetchJiraUserStories(projectKey)` - Calls `/api/jira/user-stories/:projectKey` endpoint
   - Proper error handling and response validation
3. **UI Components** (`App.tsx`):
   - **Jira Integration Section**: Added a dedicated section at the top of the form with project and user story dropdowns
   - **Project Dropdown**: Auto-populates on page load, shows "Loading projects..." while fetching
   - **User Story Dropdown**: Populates when a project is selected, shows "Loading user stories..." while fetching
   - **Auto-population**: When a user story is selected, form fields are automatically filled:
     - Story Title: Jira summary
     - Acceptance Criteria: Jira description
     - Description: Status and reporter info
     - Additional Info: Jira key reference

4. **State Management:**
   - `projects` : Array of Jira projects
   - `userStories` : Array of user stories for selected project
   - `selectedProject` : Currently selected project key
   - `selectedUserStory` : Currently selected user story ID
   - `isLoadingProjects` / `isLoadingUserStories` : Loading states for better UX
5. **User Experience Enhancements:**
   - Dropdowns are disabled appropriately (user story dropdown disabled until project selected)
   - Loading indicators in dropdown options
   - Visual separation of Jira section with background color
   - Cursor changes to indicate disabled states
6. **Styling:**
   - Added `.form-select` class for consistent dropdown styling
   - Matches existing form input styling
   - Responsive design maintained

API Endpoints Used:

- `GET /api/jira/projects` → Populates project dropdown
- `GET /api/jira/user-stories/{projectKey}` → Populates user story dropdown

Configuration Required:

Update `.env` with your actual Jira credentials:

```
JIRA_EMAIL=your-actual-email@example.com
JIRA_API_KEY=your-actual-api-key
JIRA_BASE_URL=https://your-domain.atlassian.net
```

Validation:

- ✅ TypeScript compilation successful (frontend)
- ✅ Build successful (frontend)
- ✅ Full-stack application starts correctly
- ✅ Environment variables loaded properly
- ✅ Backend server running on port 8081
- ✅ Frontend server running on port 5174

How It Works:

1. **Page Load**: Projects are automatically fetched and populate the project dropdown
2. **Project Selection**: When a project is selected, user stories for that project are fetched
3. **User Story Selection**: Selecting a user story auto-fills all form fields with Jira data
4. **Manual Override**: Users can still manually edit any field after auto-population
5. **Generate Tests**: Works exactly as before, now with Jira-sourced data

---

🎉 **Jira Integration Complete!**

The application now supports:

- ✅ Configurable Jira connectivity (email + API key)
- ✅ Auto-populated project dropdown from Jira
- ✅ Auto-populated user story dropdown for selected project
- ✅ One-click form population from selected user story
- ✅ Seamless integration with existing test generation workflow

To test the integration, update the `⚙️ .env` file with your Jira credentials and start the application with `npm run dev`. The Jira section will appear at the top of the form.